University of Royal Holloway
Lecturer:Dr. Santiago Franco
Lab Lead:Tiger-Lily Goldsmith

CS5960-2024

Lab 2.

No submission for this lab. The solutions will be published on Moodle on the second Friday morning. The last lab for this course will contain a selection of the main topics of the course and will be marked.

For this exercise sheet, we provide several Python scripts you need to work with and extend. The relevant files can be downloaded from Moodle.

/* Your code must be compatible with Python 3! */

Exercise 2: DFS vs BFS. ()

This exercise is about the analysis of the behavior of two classical search algorithms: Breadth First Search (BFS) and Depth First Search (DFS). To do so, we will be considering a 9×9 grid that a single agent needs to traverse to find the goal. The relevant Python script is **classical_search.py**. To execute it in the VM, copy it to some folder, open a terminal window in that folder, and type

python3 classical_search.py .

By executing the command, the GUI application in Figure 1 will appear in your terminal.

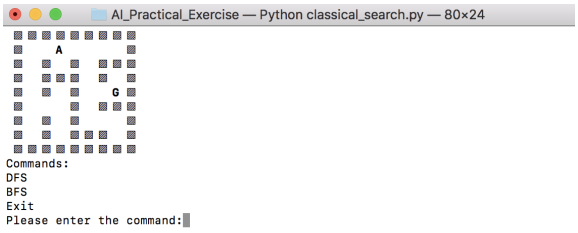


Figure 1: The commands menu and visualizer.

The provided visualizer shows where the agent is at each state, as well as the location of the goal. The agent and the goal are illustrated as A and G. Walls, i.e., non-traversable cells, are denoted by striped squares. The commands menu consists of three commands: *DFS*, *BFS*, and *Exit* which respectively execute the DFS or BFS algorithm or exit the entire program. To execute a command, type the corresponding command and press enter.

Please note that when executing a search algorithm, the program only stops and shows the commands menu again once the agent has found the goal. In this regard, if your agent doesn't move and you want to terminate the program, press Ctrl + C and start again. After your agent moves, you will see red and green dots in cells (cf. Figure 2) representing states that have already been expanded (red), or have been added to the expansion list for later expansion by the search algorithm (green).

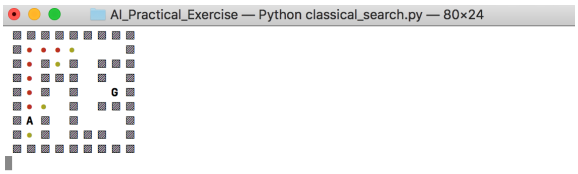


Figure 2: The agent is searching.

The **classical_search.py** script already includes implementations of DFS . To determine the applicable move actions at the agent's current position, these algorithms refer to the function ***generate_adjacent_cells***. This function is not yet fully implemented, and it is your task to complete it. The possible movements for the agent are Left, Down, Right and Up, i.e., the agent can only move into its adjacent cells in the left, bottom, right or top. ***generate_adjacent_cells*** should return all possible row and column indexes of cells adjacent (in top, bottom, left, right direction) to the agent's current position. You are also provided with an implementation of DFS. We will ask you to implement BFS. Don't forget to check if the adjacent cell is a wall or not, note that this is already done in the provided DFS implementation. Implement the following tasks:

(i) Calculate the row and column indexes of adjacent cells (top, bottom, left and right cells only).

(ii) Append the row and column indexes of adjacent cells (the ones you calculated in the previous part) to the *adjacent_cells_row*, *adjacent_cells_col* list for the expansion by the DFS or BFS algorithm. Please note that the order of appending to these lists (rows and columns) is important. This means that the ith element (adjacent cell) of each list is expanded earlier than the $(i+1)^{th}$ element by the search algorithm. The agent must expand the adjacent cells in the following order:

1. Left adjacent cell (Left move)
2. Bottom adjacent cell (Down move)
3. Right adjacent cell (Right move)

4. Top adjacent cell (Up move)

(iii) Run DFS. How many expansions until the goal is found?

(iv) Implement BFS and run it. Which algorithm finds the goal earlier, i.e. less expansions?

(v) Change the code in a way that the agent can move into diagonal cells (you can consider any orders for the expansion).

(vi) How could you reduce the number of expansions that the DFS algorithm requires to find the goal? What is the lowest number of cell expansions you could achieve with DFS until the agent finds the goal? Explain your solution.

Notes:

- You only need to change the missing sections of the code.